

Key-Value Store v5

Quorum di lettura e scrittura

Architetture dei Sistemi Distribuiti

Lezione 14

Nella lezione precedente la domanda era:

- chi e' autorizzato a scrivere?

Ora la domanda diventa:

- quante repliche devono partecipare a una scrittura?
- quante repliche devo consultare per una lettura affidabile?

Il contratto comincia a dipendere da numeri, non solo da ruoli.

Definiamo:

- N : numero totale di repliche;
- W : acknowledgment minimi per completare una scrittura;
- R : repliche minime da interrogare per completare una lettura.

Questi tre parametri descrivono il comportamento del sistema piu' di quanto faccia la sintassi dell'API.

Per scegliere il valore “piu’ recente” non basta leggere una stringa. Serve associare a ogni write:

- un valore;
- una versione.

La lettura diventa quindi:

- interrogazione di piu’ repliche;
- confronto tra versioni;
- scelta della risposta piu’ aggiornata.

Sequenza astratta:

- ① generare una nuova versione;
- ② inviare il valore alle repliche;
- ③ raccogliere almeno W acknowledgment;
- ④ dichiarare il commit della scrittura.

La write non e' piu' "scrivo su un primary". Diventa "ottengo una cardinalita' sufficiente di conferme".

Sequenza astratta:

- 1 interrogare almeno R repliche;
- 2 raccogliere coppie valore-versione;
- 3 scegliere il record con versione massima;
- 4 restituirlo al client.

Questa scelta non garantisce magia assoluta, ma costruisce un contratto molto piu' esplicito.

Condizione Importante

$$R + W > N$$

Questa disuguaglianza forza intersezione tra:

- l'insieme che ha visto l'ultima scrittura;
- l'insieme consultato dalla lettura.

E' il nucleo logico del motivo per cui i quorum forti riducono il rischio di stale read.

Con $R = 1$, $W = 1$:

- scrivere e' facile;
- leggere e' facile;
- la liveness e' buona;
- una lettura puo' fermarsi troppo presto e vedere uno stato vecchio.

Questa configurazione e' ottima per mostrare che disponibilita' e forza semantica non crescono insieme gratis.

Con $N = 3$, $R = 2$, $W = 2$:

- ogni scrittura deve toccare due repliche;
- ogni lettura consulta due repliche;
- la probabilita' di stale read cala molto;
- il costo in disponibilita' cresce.

Una stale read nasce quando:

- ① una scrittura recente raggiunge solo una parte delle repliche;
- ② la lettura consulta un sottoinsieme che non include alcuna replica fresca;
- ③ il coordinator si ferma troppo presto.

Questo e' il caso classico di configurazioni deboli come $R = 1$.

Safety:

- quorum piu' forti rendono piu' difendibile la nozione di valore piu' recente;
- riducono la possibilita' di leggere versioni troppo vecchie.

Liveness:

- quorum piu' forti falliscono piu' facilmente in presenza di nodi non raggiungibili;
- una replica in meno puo' impedire il raggiungimento di W o R .

Possiamo leggere il coordinator come automa:

- Idle -> SendToReplicas -> CollectReplies -> CheckThreshold -> Decide

La decisione finale dipende da:

- quanti reply sono arrivati;
- quali versioni sono state osservate;
- quale soglia il contratto richiede di superare.

Con i quorum, una risposta positiva non significa piu':

- “un nodo ha accettato”.

Significa piuttosto:

- “abbiamo raggiunto una cardinalita' minima di conferme”;
- “la lettura che sto restituendo e' quella con versione massima tra i quorum consultati”.

Configurazione debole:

- 1 avviare tre repliche;
- 2 usare $R = 1$, $W = 1$;
- 3 introdurre disponibilit  parziale;
- 4 osservare una lettura che non vede l'ultima versione.

Configurazione forte:

- ① usare $R = 2$, $W = 2$;
- ② ripetere l'esperimento;
- ③ osservare il miglioramento di consistenza;
- ④ osservare il costo in disponibilit .

In questo laboratorio una nuova versione viene prodotta da un coordinator sotto controllo. Ma in generale restano aperti:

- writer concorrenti;
- versioni concorrenti;
- ordini parziali;
- riconciliazione di conflitti.

Questa tappa rafforza molto il contratto, ma non esaurisce ancora il problema della coerenza distribuita.

Con i quorum il contratto del sistema diventa una proprietà numerica:

- quante repliche servono per scrivere;
- quante servono per leggere;
- quale intersezione minima deve esistere tra i due insiemi.

La stessa API può restare quasi immutata, ma il suo significato dipende ora da cardinalità e versioni.